



(12) 发明专利申请

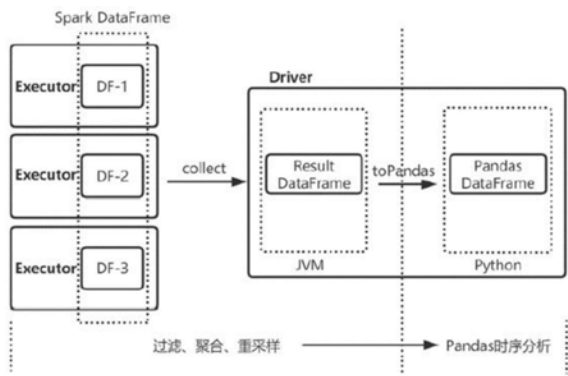
(10) 申请公布号 CN 112612823 A
(43) 申请公布日 2021. 04. 06

(21) 申请号 202011467348.3
(22) 申请日 2020.12.14
(71) 申请人 南京铁道职业技术学院
地址 210000 江苏省南京市浦口区珍珠南路65号
(72) 发明人 黄必栋
(74) 专利代理机构 西安研创天下知识产权代理
事务所(普通合伙) 61239
代理人 陈明星
(51) Int.Cl.
G06F 16/2458 (2019.01)
G06F 16/27 (2019.01)

权利要求书1页 说明书7页 附图3页

(54) 发明名称
一种基于PySpark和Pandas融合的大数据时序分析方法

(57) 摘要
本发明公开了一种基于PySpark和Pandas融合的大数据时序分析方法,其使用Spark对大规模数据进行过滤、聚合等变换操作,通过降采样方法把大规模时序数据转化为小规模均匀等距时序数据,再使用PySpark的toPandas方法转化为Pandas DataFrame,最后使用Pandas库提供的时序分析算法进行时序分析。实际应用中具有较好的运行效率和性能。



1. 一种基于PySpark和Pandas融合的大数据时序分析方法,其特征在于,包括以下步骤:

步骤1:通过数据采集模块抓取来海量数据;

步骤2:利用Spark对海量数据进行补全、统一时间字段等格式标准化操作,得到大规模时序数据。

步骤3:通过过滤、聚合等变换操作把大规模时序数据降采样为小规模均匀等距时序数据;

步骤4:基于PySpark将得到的小规模均匀等距时序数据转化为Pandas数据;

步骤5:利用Pandas库中的时序分析函数对得到的Pandas数据进行时序分析,从而得到时序数据。

2. 根据权利要求1所述的一种基于PySpark和Pandas融合的大数据时序分析方法,其特征在于,步骤3的具体操作步骤包括:

步骤31:待处理时序数据中的每条记录为事件记录信息,其包括编号id、时间戳timestamp、事件名event以及相应的值value;

步骤32:通过时间范围(t_start,t_end)和事件名筛选出事件A对应的Spark DataFrame df_A;

步骤33:为df_A添加粗粒度时间点列new_time,并对此列进行分组,使用均值函数avg对value列进行聚合计算;

步骤34:根据时间范围(t_start,t_end)和采样粒度生成采样时间点DataFrame ts_df;

步骤35:通过ts_df和df_new_time进行左外连接,对缺失时间点进行补全,并且缺失时间点的value值设为0;

步骤36:最终得到事件A在(t_start,t_end)时间范围内的时序数据[tm,value],其中,tm为粗粒度时间点,value为粗粒度时间间隔内的相应值的均值。

3. 根据权利要求1所述的一种基于PySpark和Pandas融合的大数据时序分析方法,其特征在于,步骤4所述的转化方法为:使用PySpark中的toPandas方法将Spark DataFrame转换为Pandas DataFrame。

4. 根据权利要求3所述的一种基于PySpark和Pandas融合的大数据时序分析方法,其特征在于,步骤5的具体操作步骤为:根据得到的Pandas DataFrame设置时间字段,再利用Pandas中的时序分析函数得到Pandas时序数据。

一种基于PySpark和Pandas融合的大数据时序分析方法

技术领域

[0001] 本发明属于大数据分析技术领域,具体涉及一种基于PySpark和Pandas融合的大数据时序分析方法。

背景技术

[0002] Python和R语言是数据分析中的主流编程语言,由于Python的Pandas库的流行,Python成为数据分析的热门编程语言。使用Pandas进行数据分析适用于单机小规模数据分析场景,无法适应大规模数据处理和计算要求。Spark则是大数据处理和迭代计算的主流计算平台,支持Python语言,PySpark即是SparkAPI的Python语言接口。

[0003] 原生的Spark在时序分析方面提供了较少的时序分析功能和算法,虽然第三方库spark-timeseries提供了基于Spark时间序列分析算法库,但是其从功能和生态上看并不及Pandas库的时间序列分析模块。而Pandas库只适用于单机小规模数据量的处理,无法运行于分布式系统环境中以处理大规模数据。

[0004] 随着信息化产业的不断发展,大型流程工业企业在生产信息化过程中会产生越来越多的海量历史时序数据。因此对时序数据的处理规模、处理速度提出了更高的要求。但是目前的大数据处理的平台Spark在时序分析上提供了较粗粒度和有限的时序分析方法,而当前主流的时序分析库Pandas虽然提供了丰富的时序分析算法工具包,但是只能单机运行,无法适应大规模时序数据的处理。因此,如何高效的对大规模数据应用时序分析算法进行时序分析是目前亟待解决的问题。

发明内容

[0005] 针对上述存在的问题,本发明提供一种基于PySpark和Pandas融合的大数据时序分析方法,使用PySpark结合Pandas进行时序分析,提高大数据分析效率。

[0006] 实现本发明目的的技术解决方案为:

[0007] 一种基于PySpark和Pandas融合的大数据时序分析方法,其特征在于,包括以下步骤:

[0008] 步骤1:通过数据采集模块抓取来海量数据;

[0009] 步骤2:利用Spark对海量数据进行补全、统一时间字段等格式标准化操作,得到大规模时序数据。

[0010] 步骤3:通过过滤、聚合等变换操作把大规模时序数据降采样为小规模均匀等距时序数据;

[0011] 步骤4:基于PySpark将得到的小规模均匀等距时序数据转化为Pandas数据;

[0012] 步骤5:利用Pandas库中的时序分析函数对得到的Pandas数据进行时序分析,从而得到时序数据。

[0013] 进一步地,步骤3的具体操作步骤包括:

[0014] 步骤31:待处理时序数据中的每条记录均为事件记录信息,其包括编号id、时间戳

timestamp、事件名event以及相应的时间值value;

[0015] 步骤32:通过时间范围(t_start,t_end)和事件名筛选出事件A对应的Spark DataFrame df_A;

[0016] 步骤33:为df_A添加粗粒度时间点列new_time,并对此列进行分组,使用均值函数avg对value列进行聚合计算;

[0017] 步骤34:根据时间范围(t_start,t_end)和采样粒度生成采样时间点DataFrame ts_df;

[0018] 步骤35:通过ts_df和df_new_time进行左外连接,对缺失时间点进行补全,并且缺失时间点的value值设为0;

[0019] 步骤36:最终得到事件A在(t_start,t_end)时间范围内的时序数据[tm,value],其中,tm为粗粒度时间点,value为粗粒度时间间隔内的相应值的均值;

[0020] 进一步地,步骤4所述的转化方法为:使用PySpark中的toPandas方法将Spark DataFrame转换为Pandas DataFrame。

[0021] 进一步地,步骤5的具体操作步骤为:根据得到的Pandas DataFrame设置时间字段,再利用Pandas中的时序分析函数得到Pandas时序数据。

[0022] 本方法与现有技术相比,具有以下有益效果:

[0023] 本发明从设计上充分考虑了用户使用习惯、数据处理性能将PySpark数据分析方法和Pandas数据分析方法进行结合,使得已有的时序分析方法能应用于大规模数据的处理,提高了大规模时序数据的分析效率和速度,在实际应用中具有较好的运行效率和性能。

附图说明

[0024] 图1为PySpark的架构;

[0025] 图2为基于ApacheArrow的Pandas UDF的架构;

[0026] 图3为用Apache Arrow进行Spark DataFrame和Pandas DataFrame的转换的示意图;

[0027] 图4为本发明PySpark和Pandas库配合进行时序分析的过程图;

[0028] 图5为本实施例中通过源ip统计出的公司数和告警数结果的示意图;

[0029] 图6为本实施例中源地址为203.171.228.159的分析结果的示意图;

[0030] 图7为本实施例中源地址为203.171.228.159的告警数据转为Pandas时序数据后进行Dickey-Fuller单位根检验的结果示意图;

具体实施方式

[0031] 为了使本领域的普通技术人员能更好的理解本发明的技术方案,下面结合附图和实施例对本发明的技术方案做进一步的描述。

[0032] 1、PySpark架构和原理

[0033] 1.1基本架构

[0034] Spark使用Scala语言开发,运行于JVM之上,比较容易提供Scala或Java接口,而Python程序与JVM不兼容,较难提供Python接口。Spark为了提供Python接口,同时又能兼顾数据处理性能,Spark 0.7版本引入了PySpark,且PySpark的架构如附图1所示。

[0035] 从附图1中可以看出,PySpark的Driver端由Python进程和JVM进程构成,他们之间通过进程间通讯方法进行接口调用和数据交换。PySpark基于Spark Java接口实现,通过使用Py4j,使得Python代码能够调用Java代码。Executor端主要是基于JVM,若使用了PythonUDF,则通过启动Python进程调用UDF。

[0036] 1.2PythonUDF的性能影响

[0037] 若未使用PythonUDF,由于数据处理主要运行在Executor端,Executor不需要启动Python进程以进行UDF调用,PySpark的数据处理的性能并没有因为Python的引入而下降。若使用Python UDF,Executor端通过启动Python进程调用UDF,JVM进程和Python进程的数据交换需经过序列化操作和管道传输,加上Python代码的解释执行,PySpark的性能会有一定的下降。针对使用Python UDF带来的性能问题,Spark在2.3版本中引入了Pandas UDF,通过Apache Arrow提供的基于内存的列式存储数据层,减少JVM进程和Python进程数据交换过程中的序列化操作,从而提高了性能,其架构如附图2所示。

[0038] Pandas UDF也称之为Vectorized UDF,即支持向量化处理的UDF,相较于早期的一次UDF执行处理单条记录,改为一次UDF执行处理多条数据,UDF的执行性能得到进一步提高。Spark 3.0对Pandas UDF继续进行了改进,解决了Pandas UDF易用性差的问题,利用Python 3.6提供的Python type hints功能,使Pandas UDF定义清晰,易于使用。

[0039] 1.3PySpark与Pandas库的集成

[0040] Pandas库是Python生态中数据分析的主要工具,已成为数据分析工具的标准。Pandas库和Python数据科学生态中的numpy、matplotlib、scikit-learn等库紧密集成,能适应多种数据处理和分析场景,包括:基本的统计分析、数据补全、时序分析等。

[0041] Pandas库通常在单机上使用,受制于内存和CPU的限制,处理的数据量和性能都有限。而Spark则能运行于易扩展的分布式系统中,能充分利用系统的计算和内存资源,适合处理海量数据并具有较好的性能。但是PySpark和Pandas库还存在一些差异,功能和生态都不及Pandas库,数据分析师较难迁移到PySpark。融合Pandas库是PySpark目前发展的主要方向,使PySpark既有类似Pandas的API,同时又能高性能地处理海量数据。目前在PySpark中使用PandasAPI有两种方法:

[0042] (1) PySpark和Pandas库混合使用

[0043] PySpark支持Spark DataFrame和Pandas DataFrame之间的相互转换,示例代码如下:

```
[0044] import pandas as pd
[0045] # 创建一个Pandas DataFrame
[0046] pdf = pd.DataFrame(np.random.rand(100, 3))
[0047] # 从Pandas DataFrame 创建一个Spark DataFrame
[0048] df = spark.createDataFrame(pdf)
[0049] # 把Spark DataFrame 转成Pandas DataFrame
[0050] result_pdf = df.select("*").toPandas()
```

[0051] 实际使用中混合方法一般先使用Spark进行大数据量的处理,把大规模数据转成小规模数据,再转换为Pandas DataFrame,从而进一步使用Pandas API进行分析。toPandas()方法实际上是一种Spark collect action,数据处理结果先收集到Driver端,再进行转

换。处理结果由Driver端的JVM进程传至Python进程,若处理结果比较大,使用本地网络通讯方式传递数据的性能不佳。与Pandas UDF的性能改进方法类似,Spark使用Apache Arrow解决Spark DataFrame和Pandas DataFrame的转换中跨进程数据交换性能问题,其设计如附图3所示。此功能在Spark 3.0中默认是关闭的,需要设置`spark.sql.execution.arrow.pyspark.enabled`为true打开。

[0052] (2) 使用KoalasAPI

[0053] Koalas API的目标是在Spark上提供类似Pandas API。Spark3.0中提供了Koalas 1.0版本,已实现了60%~80%的Pandas功能。Koalas包的下载非常活跃,从2020年1月至2020年6月,每天有2~3万的下载量。

[0054] 上述的混合使用方法和Koalas方法目前都还不完善,并不能完全解决Pandas的融合问题。首先,混合使用方法对使用者要求较高,使用者既要熟悉PySpark,又要熟悉Pandas。Spark DataFrame与Pandas DataFrame的转换有一定的局限性,Spark DataFrame若规模过大,则可能由于Driver端的内存不足而导致转换失败。其次,KoalasAPI由于设计上的差异,不存在内存限制问题,但还不能完全兼容Pandas API。

[0055] 2、PySpark的安装和运行方法

[0056] PySpark的安装既可以通过下载Spark安装包,也可以使用Python的包管理器pip或conda安装。使用包管理器较为方便,包管理器会下载好完整的Spark程序,安装和升级非常方便。

[0057] PySpark的运行有交互式运行和提交代码执行两种方式,前者适用于数据分析场景,后者适用于数据批处理场景。Spark自带的pyspark shell为交互式编程工具,spark-submit脚本提供了提交代码执行的方法。由于Notebook编程方式基于Web,可以更方便、灵活地进行代码编辑和数据可视化,实际使用中交互式分析大多选择Notebook。在Notebook中启动Spark的方式与spark-submit脚本不同,Spark的配置参数并不是通过设置命令行参数,而是通过代码指定。

[0058] 以运行PySpark于Yarn集群为例,以下代码提供了一个Spark启动函数,该函数返回SparkSession对象。其运行的示例代码如下:

```
1  def start_spark_yarn(executor_num,core_num,
2                          executor_mem,driver_mem,
3                          app_name,pkgs,hadoop_conf_dir):
4      if 'HADOOP_CONF_DIR' not in os.environ:
5          os.environ['HADOOP_CONF_DIR'] =\
6              hadoop_conf_dir
7      conf = SparkConf()\
8          .set('spark.executor.instances', executor_num)\
[0059] 9          .set('spark.executor.cores', core_num)\
10         .set('spark.executor.memory', executor_mem)\
11         .set('spark.driver.memory', driver_mem)
12     for pkg in pkgs:
13         conf = conf.set('spark.jars.packages', pkg)
14     conf.setMaster('yarn')
15     sc = SparkContext(conf=conf)
16     spark = SparkSession.builder\
[0060] 17         .appName(app_name).getOrCreate()
18     return spark
```

[0061] 以上代码中的第4-6行代码设置了Hadoop配置文件目录环境变量,用于寻找Yarn的配置文件;第7-11行代码设置了Executor数量、核数、内存等运行参数;第12-13行代码设置了所依赖的jar包;第14行代码指定了以Yarn模式运行。调用该函数返回SparkSession对象后,即可调用PySpark接口。

[0062] 3、基于PySpark和Pandas库进行时序分析

[0063] 由于在PySpark编程中能同时使用SparkAPI和Pandas库,并且Spark DataFrame和Pandas DataFrame可以进行双向转换,PySpark和Pandas库可以相互配合,完成时序分析的不同阶段的不同功能,最终达到既能处理大规模数据又能应用丰富的时序分析算法的目的。

[0064] PySpark配合Pandas库进行时序分析的设计如附图4所示,其基本思路是使用Spark对大规模数据进行过滤、聚合等变换操作,通过重采样(降采样)方法把大规模时序数据转化为小规模均匀等距时序数据,再使用PySpark的toPandas方法转化为Pandas DataFrame,最后使用Pandas库提供的时序分析算法进行时序分析。

[0065] 在Spark中对大规模时序数据降采样是整个过程的关键,下面通过实例阐述进行时序分析的具体步骤和方法。令待处理时序数据为事件记录信息,其Schema如表1所示。其中timestamp字段为事件发生的时间戳,单位为秒;event为事件名称,为枚举类型;value字段记录了事件中对应的值(为了简化问题,只定义一个值为代表)。

[0066] 表1待处理时序数据的Schema

[0067]

字段	类型	备注
id	string	编号
timestamp	bigint	单位为秒的时间戳

event	string	事件名
value	bigint	时间值

[0068] 现需要对数据降采样为:事件A在某个时间范围(t_start, t_end)内的时序数据 $[tm, value]$, tm 为整小时点, $value$ 为单位小时内的事件值的均值。

[0069] 时序分析的步骤如下:

[0070] (1) 通过时间范围和事件筛选出事件A的DataFrame df_A 。

#df 为全量数据

$df_A = df.filter(df.event='A')$

[0071] $.filter(df.timestamp \geq t_start)$

$.filter(df.timestamp \leq t_end)$ $ts_df =$

$spark.createDataFrame(ts, ['tm'])$

[0072] (2) 为 df_A 添加整小时点列: new_time (该值根据 $timestamp$ 字段计算), 并对此列进行分组, 使用均值函数 avg 对 $value$ 列进行聚合计算。

$df_tmp = df_A.withColumn('new_time', F.floor(df_A.timestamp/(3600))*3600)$

$df_new_time = df_tmp.groupBy('new_time')$

[0073] $.agg(F.avg('value').alias('value'))$

$.sort(F.asc('new_time'))$

[0074] (3) 根据时间范围和采样粒度(粗粒度)生成采样时间点DataFrame ts_df 。

ts_df 包括一列: tm , tm 为时间范围内的整小时点。

$ts = []$

[0075] $t_min = \text{math.floor}(t_start/3600)*3600$ $t_max =$

$\text{math.floor}(t_end/3600)*3600$ for n in $\text{range}(t_min, t_max+1, 3600)$:

$ts.append((n,))$

[0076] (4) 通过 ts_df 和 df_new_time 进行左外连接, 对缺失时间点进行补全, 缺失时间点的 $value$ 值设为0。

[0077] $df_final = ts_df.join(df_new_time, ts_df.tm == df_new_time.new_time, 'left_outer')$

[0078] $.select('tm', 'value').na.fill(0).sort(F.asc('tm'))$

[0079] (5) 把Spark DataFrame转换为Pandas time series。

#把Spark DataFrame 转为Pandas DataFrame

$df_p = df_final.toPandas()$ #设置时间字段

$df_p['timestamp'] = pandas.to_datetime(df_p['tm'], unit='s')$

[0080] $df_result = df_p[['timestamp', 'value']]$

$.set_index('timestamp')$ #转为 time series

$tm_series = df_result['value']$

[0081] 上述方法中步骤1和2降低了数据量, 步骤1通过事件名和时间范围过滤筛选数据, 步骤2按粗粒度的时间单位进行聚合再次降低了数据量。步骤3和4用于对缺失的时间点进行补全, 最后在步骤5中调用 $toPandas()$ 函数把Spark DataFrame转至Pandas DataFrame,

进一步转化为Pandas time series。

[0082] 实施例

[0083] 在本实施例中以某安全云平台告警信息进行时序分析,该安全云平台汇总了多家公司的安全告警事件数据,这些数据实时的从公司本地的安全产品端收集到云端,进行汇聚存储,并且数据涉及到每个公司的所有的安全告警事件,数据量较大。

[0084] 由于安全分析的特性,需要进行全局的跨公司的数据分析,特别是安全事件的时序分析,例如某个攻击者ip对各家公司的攻击时间规律、行业中某种类型的攻击趋势等。然而这些类型的时序分析如果仅仅依靠Spark平台是较难完成的,利用本申请提出的方法则能够高效、高性能的完成时序分析任务。

[0085] 下表2为一条安全告警事件的数据格式。

[0086] 表2安全告警事件的数据格式

[0087]	字段名	含义
	alarm_id	告警编号
	timestamp	告警发生的时间
	alarm_name	告警名
	company	所属公司
	src_addr	源ip地址
	dst_addr	目的ip地址
	domain	域名
	alarm_content	告警内容

[0088] 基于表2中的告警记录,对攻击者的攻击时间规律进行时序分析,具体步骤为:

[0089] 步骤1:使用PySpark统计出某个源ip地址关联的公司数和告警数总和的Top 10,这些ip地址分布在多个公司和多个告警之中,为高可疑ip地址,统计出的结果如附图5所示。

[0090] 步骤2:把步骤1分析出的Top 10源地址到告警数据中进行过滤和统计,通过本申请提出的方法转换为Pandas时序数据,再进行Dickey-Fuller单位根检验测试其平稳性,其检测结果如附图7所示;并且其中源地址为203.171.228.159的分析结果如附图6所示。

[0091] 通过分析结果可以看出,本发明提出的方法能够对攻击者的攻击时间进行有效地时序分析,提高了大规模时序数据的分析效率和速度,通过Dickey-Fuller单位根检验结果可知该方法时序处理结果稳定。

[0092] 本说明书中未作详细描述的内容属于本领域专业技术人员公知的现有技术。尽管参照前述实施例对本发明专利进行了详细的说明,对于本领域的技术人员来说,其依然可以对前述各实施例所记载的技术方案进行修改,或者对其中部分技术特征进行等同替换,凡在本发明的精神和原则之内,所作的任何修改、等同替换、改进等,均应包含在本发明的保护范围之内。

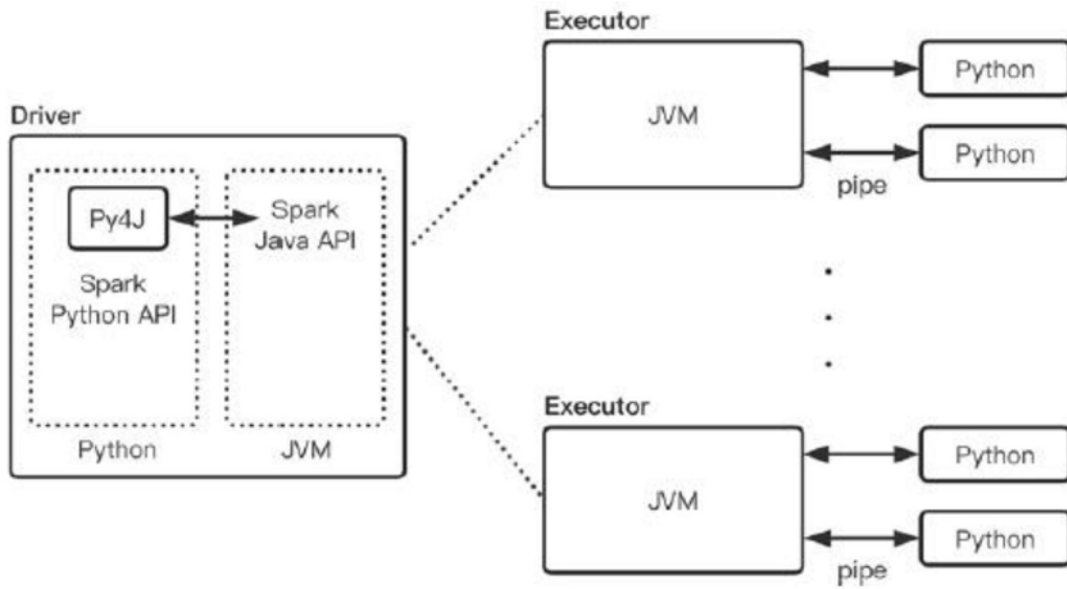


图1

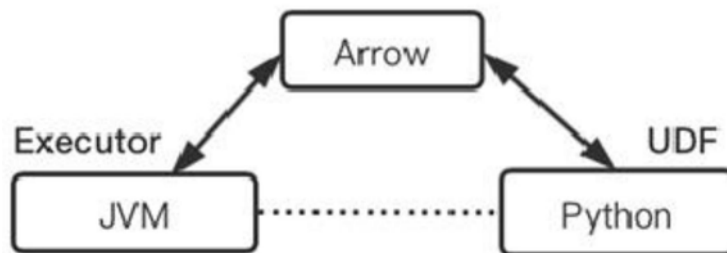


图2

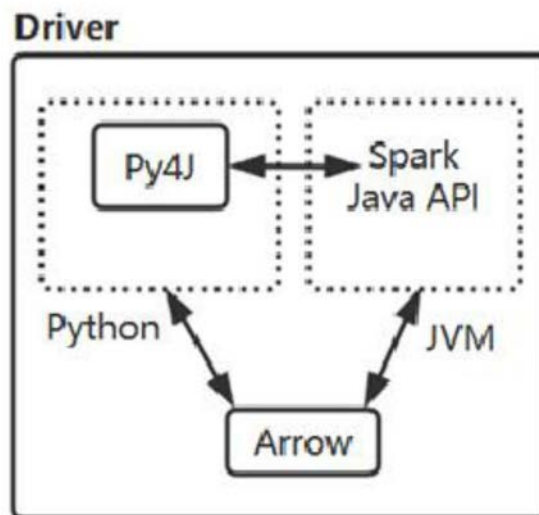


图3

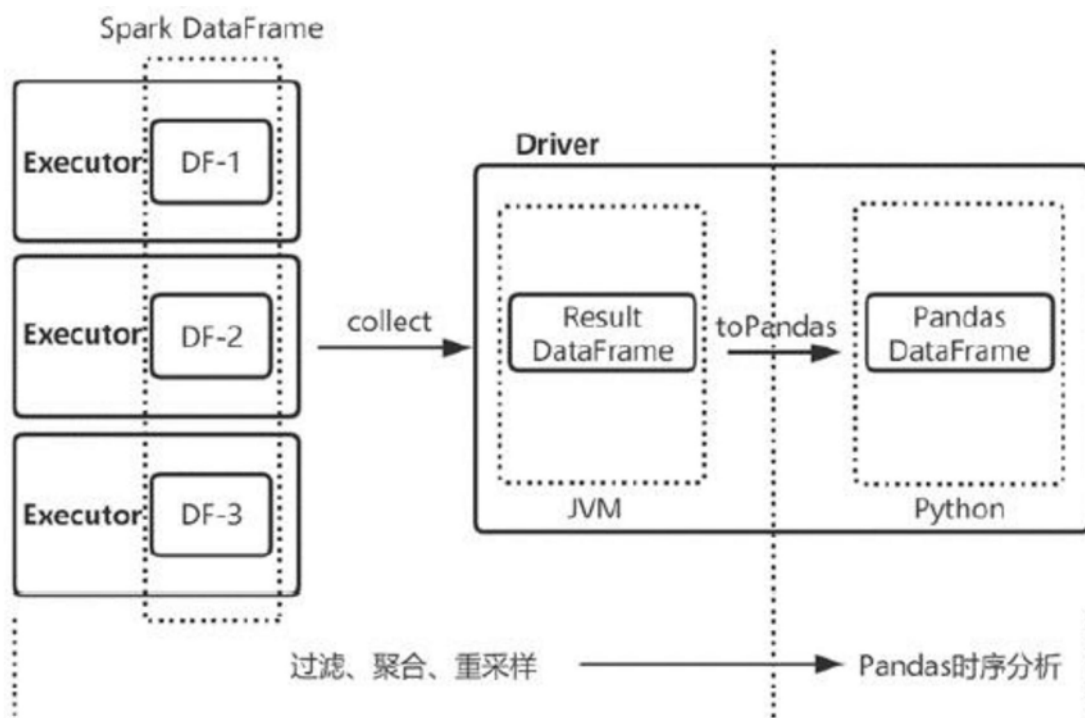


图4

src_addr	company_count	alarm_count	sum_count
111.206.52.98	5	51	56
203.171.228.159	15	40	55
116.196.19.207	14	38	52
106.120.161.64	4	46	50
221.122.70.6	3	46	49
210.52.224.30	5	42	47
221.122.70.2	3	44	47
94.191.43.211	13	34	47
210.52.224.29	5	41	46
210.52.224.23	5	41	46

only showing top 10 rows

图5

【ioc:203.171.228.159为源地址分析:】

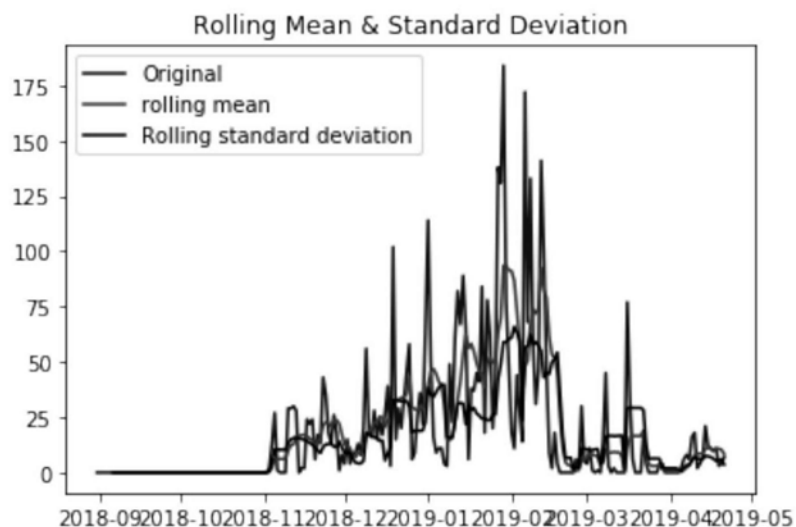
>>在40次告警中出现

>>详情如下:

alarm_name	count
网络攻击告警	1102
HTTP信道攻击-报文协议异常	1020
流量检测 Web攻击 - 通用Web攻击	399
自动化工具web攻击 (http协议违背)	385
疑似针对单台服务器的DDoS攻击-HTTP流	342
流量检测 入侵检测告警	257
发生WEB攻击-跨站脚本	244
发生WEB攻击-SQL注入	96
发生WEB攻击-webshell攻击	88

图6

{'src_addr': '203.171.228.159'}



Results of Dickey-Fuller Test:

Test Statistic -1.548469
 p-value 0.509413
 #Lags Used 13.000000
 Number of Observations Used 220.000000
 Critical value (1%) -3.460428
 Critical value (5%) -2.874769
 Critical value (10%) -2.573821
 dtype: float64

图7